

Broadcasting, Worked Out by Hand

How mismatched shapes combine without copying

A complete numerical walk-through of *broadcasting* — the rule that lets PyTorch and NumPy add, multiply, and compare tensors of *different* shapes. We align shapes from the right, decide compatibility dimension by dimension, and watch a size-1 axis get *virtually stretched* to fit. Nothing is hand-waved: every number below was produced by the NumPy/PyTorch reference program in Appendix A, so the arithmetic is exact and self-consistent — including the stride trick that makes the stretch cost *zero* bytes.

What this document does. It takes four tiny shape pairs — a column plus a row, a per-channel image scale, a bias added to a batch, and a deliberately broken pair — and runs each through the broadcasting rule one dimension at a time. For every successful case we show *exactly* which two entries combine into each output cell, and for the failing case we show *why* the rule rejects it.

How to read this. Each example is small enough to verify in your head, and each gets a coloured box isolating one idea. The crucial mechanical fact — that broadcasting sets the stretched axis's *stride* to 0 rather than copying data — ties directly back to the stride model from the Tensors topic, so keep that (shape, stride) recipe in mind.

Concepts covered, in order: right-alignment of shapes · the two compatibility rules (equal, or one is 1) · left-padding with 1s · the output rule $\text{out} = \max(a, b)$ · the $(3, 1) + (1, 4)$ outer-sum · a $(2, 2, 3) \times (3,)$ per-channel scale · a $(4, 3) + (3,)$ bias add · the failing $(3,) + (4,)$ case · the stride-0 memory trick · the silent $(n, 1)$ vs $(1, n)$ outer-product trap.

1 The rule, stated once and for all

Broadcasting is the answer to a single question: *when the two operands of an elementwise op have different shapes, can we still pair up their elements — and if so, how?* The whole rule fits in three lines. Write both shapes as tuples, then:

Align	line the shapes up by their <i>rightmost</i> dimension.
Pad	the shorter shape gets 1s prepended on the <i>left</i> .
Compare	each aligned pair (a, b) is compatible iff $a = b$ or one of them is 1.

If every aligned pair passes, the operation succeeds and the output dimension on that axis is

$$\boxed{\text{out_dim} = \max(a, b)} \quad (\text{legal only because the smaller value is always 1 or equal}).$$

A size-1 (or padded) axis is then *virtually stretched* from 1 up to `out_dim`: the *same* value is reused along that axis. No data is copied — Section 6 shows the stretch is implemented by setting that axis's stride to 0.

Intuition

“Stretching” a size-1 axis just means *reusing the one value you have* for every position along that axis. A single bias vector serves all rows; a single per-channel scale serves every pixel. Broadcasting is the compiler noticing “you only gave me one of these, so you must mean the same one everywhere” — and arranging the memory reads so that one value is fetched again and again, instead of materialising a fat copy.

2 Step 1 — A column meets a row: $(3, 1) + (1, 4) \rightarrow (3, 4)$

This is the purest case: *both* operands have a size-1 axis, and each stretches to cover the other. Take a column a and a row b :

$$a = \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix} (3, 1), \quad b = [1 \ 2 \ 3 \ 4] (1, 4).$$

Align from the right and compare each pair:

$$\begin{array}{llll} \text{axis 0 : } 3 \text{ vs } 1 & \Rightarrow & \text{one is } 1 \checkmark & \rightarrow \max = 3 \\ \text{axis 1 : } 1 \text{ vs } 4 & \Rightarrow & \text{one is } 1 \checkmark & \rightarrow \max = 4 \end{array} \quad \Longrightarrow \quad \text{output shape } (3, 4).$$

a 's lone column is reused across all 4 output columns; b 's lone row is reused across all 3 output rows. Output cell (i, j) is therefore $a_i + b_j$:

$$C = a + b = \begin{bmatrix} 11 & 12 & 13 & 14 \\ 21 & 22 & 23 & 24 \\ 31 & 32 & 33 & 34 \end{bmatrix}.$$

Reading off three cells to see exactly which inputs combined:

$$\begin{aligned} C_{0,0} &= a_0 + b_0 = 10 + 1 = 11, \\ C_{1,2} &= a_1 + b_2 = 20 + 3 = 23, \\ C_{2,3} &= a_2 + b_3 = 30 + 4 = 34. \end{aligned}$$

Every input number a_i appears in a whole row of C ; every b_j appears in a whole column. That “each input touches a whole line of the output” pattern is the visual signature of a stretched axis.

Mini-example — this operation in isolation

The output shape is computed per axis with `max`, but only because the loser is always a 1. On axis 0 we had $(3, 1) \rightarrow \max = 3$; on axis 1 we had $(1, 4) \rightarrow \max = 4$. If instead the pair had been 3 vs 4 (no 1 in sight), `max` would be meaningless and the op would *error* — which is precisely Step 4.

3 Step 2 — Per-channel scale: $(2, 2, 3) \times (3,) \rightarrow (2, 2, 3)$

A colour image is a tensor of shape (height, width, channels). To brighten the red channel, dim the blue, and leave green alone, you want *one* multiplier per channel — a vector of length 3 — applied at every pixel. Use a 2×2 stand-in image filled with $0, \dots, 11$ and the channel scale $s = [0.5, 1.0, 2.0]$:

$$\text{img}[0] = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}, \quad \text{img}[1] = \begin{bmatrix} 6 & 7 & 8 \\ 9 & 10 & 11 \end{bmatrix}, \quad s = [0.5 \quad 1.0 \quad 2.0].$$

Align the shapes from the right. The 1-D scale is *shorter*, so it is left-padded with 1s up to rank 3:

$$\begin{array}{rcl} \text{img:} & 2 & 2 \quad 3 \\ s: & 1 & 1 \quad 3 \quad \leftarrow \text{padded } (3,) \rightarrow (1, 1, 3) \\ \hline \text{max:} & 2 & 2 \quad 3 \end{array}$$

The two padded axes (size 1) stretch to 2 and 2: the *same* three numbers $[0.5, 1.0, 2.0]$ multiply every one of the 2×2 pixels. Output cell $(i, j, c) = \text{img}_{i,j,c} \cdot s_c$. Two pixels worked out:

$$\begin{aligned} \text{out}[0, 0] &= [0, 1, 2] \odot [0.5, 1.0, 2.0] = [0, 1, 4], \\ \text{out}[1, 1] &= [9, 10, 11] \odot [0.5, 1.0, 2.0] = [4.5, 10, 22]. \end{aligned}$$

The full result:

$$\text{out}[0] = \begin{bmatrix} 0 & 1 & 4 \\ 1.5 & 4 & 10 \end{bmatrix}, \quad \text{out}[1] = \begin{bmatrix} 3 & 7 & 16 \\ 4.5 & 10 & 22 \end{bmatrix}.$$

When this is the right choice

Reach for a bare $(C,)$ vector against an $(\dots, C)$ tensor whenever the operation is “one parameter per channel/feature applied everywhere else”: per-channel scale, per-feature mean/std in normalisation, a per-class weight. The right-alignment rule makes the channel axis line up automatically, and the leading axes stretch for free. This is exactly the `(X - mu) / sigma` feature-normalisation pattern from the course, where `mu` and `sigma` are $(5,)$ and `X` is $(100, 5)$.

4 Step 3 — Bias add over a batch: $(4, 3) + (3,) \rightarrow (4, 3)$

The everyday case in a linear layer: a batch of 4 vectors, each of length 3, plus a single bias vector of length 3 added to *every* row. Let

$$A = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \\ 9 & 10 & 11 \end{bmatrix} (4, 3), \quad \text{bias} = [100 \ 200 \ 300] (3,).$$

Right-align: the bias is rank 1, so it pads to $(1, 3)$. Axis 0 is 4 vs 1 ($\rightarrow \max 4$, bias stretches down all 4 rows); axis 1 is 3 vs 3 (equal). Output $(4, 3)$. Each output row is $A_i + \text{bias}$:

$$R = A + \text{bias} = \begin{bmatrix} 100 & 201 & 302 \\ 103 & 204 & 305 \\ 106 & 207 & 308 \\ 109 & 210 & 311 \end{bmatrix}, \quad \begin{aligned} R_0 &= [0, 1, 2] + [100, 200, 300] = [100, 201, 302], \\ R_3 &= [9, 10, 11] + [100, 200, 300] = [109, 210, 311]. \end{aligned}$$

The one bias vector is reused four times; the column structure $+100, +200, +300$ is visible in every row.

Intuition

Notice that Steps 2 and 3 are the *same* pattern at different ranks: a length- C vector broadcast against a tensor whose last axis is C . The leading axes — whether that's a batch dimension $(4, \cdot)$ or both spatial dimensions $(2, 2, \cdot)$ — are the ones padded to 1 and stretched. Learn this one shape and you have learned bias addition, feature scaling, and per-channel normalisation all at once.

5 Step 4 — When it fails: $(3,) + (4,)$, and exactly why

Not every pair broadcasts. Add a length-3 vector to a length-4 vector:

$$[1, 2, 3] + [1, 2, 3, 4] = ?$$

Both are rank 1, so there is nothing to pad. Align from the right and compare the single axis: 3 vs 4. They are *not* equal, and *neither* is 1. The rule has no way to pair a 3-long axis with a 4-long axis, so the operation is rejected. PyTorch raises

The size of tensor a (3) must match the size of tensor b (4) at non-singleton dimension 0

and NumPy raises

operands could not be broadcast together with shapes (3,) (4,)

The phrase *non-singleton dimension* is the rule restated: a singleton (size 1) axis would have been allowed to stretch, but 3 and 4 are both non-singletons, so they must match exactly, and they don't.

Watch out

The fix is *not* to pad with zeros or truncate — broadcasting never invents or drops data. Decide which axis you actually meant the vectors to lie along and reshape one of them to a 1 there: `a[:,None]` makes $(3,) \rightarrow (3,1)$, and then $(3,1) + (4,)$ aligns as $(3,1)$ vs $(1,4) \rightarrow (3,4)$, the outer sum of Step 1. The error is telling you a real ambiguity exists; resolve it by stating the axis explicitly.

6 The compatibility table at a glance

Every per-axis decision reduces to one of three outcomes. This is the entire rule as a lookup table:

dim a	dim b	compatible?	result $\max(a,b)$ / reason
3	3	yes (equal)	3
3	1	yes (stretch b)	3
1	4	yes (stretch a)	4
1	1	yes (both stay)	1
5	1	yes (stretch b)	5
2	2	yes (equal)	2
3	4	NO	error: neither is 1

Read it top to bottom: the only way to fail is the last row — two different sizes with neither equal to 1. Everything else broadcasts, and the output size is always the larger of the two (which is safe precisely because the smaller is 1 or they are equal).

7 Step 5 — The memory trick: stretching is stride 0

Here is the part that makes broadcasting *cheap*. Recall from the Tensors topic that a tensor is a flat run of numbers plus a *stride* vector telling you how many elements to skip to advance one step along each axis. Broadcasting a size-1 axis up to size N does **not** copy the data N times. Instead it sets that axis's stride to 0:

$$\boxed{\text{stretched axis} \implies \text{stride} = 0}$$

A stride of 0 means “advancing along this axis does not move in memory” — so every index along it reads the *same* underlying element. Watch the bias vector from Step 3, $b = [1, 2, 3, 4]$ of shape $(4,)$ and stride (1) , broadcast to $(3,4)$:

$$\text{shape } (4), \text{ stride } (1) \xrightarrow{\text{broadcast to } (3,4)} \text{shape } (3,4), \text{ stride } (0, 1).$$

The new leading axis has stride 0: rows 0, 1, 2 all point at the *same* four numbers. For the column a of Step 1, shape $(3,1)$ with stride $(1,1)$, broadcasting to $(3,4)$ gives stride $(1, 0)$ — the *column* axis is the one held still:

$$a : (3,1) \text{ stride } (1,1) \rightarrow (3,4) \text{ stride } (1,0), \quad b : (1,4) \text{ stride } (4,1) \rightarrow (3,4) \text{ stride } (0,1).$$

Both expanded views occupy their original 3 and 4 numbers in memory; the $(3,4)$ result is computed by reading each through its stride-0 axis. That is why a per-channel scale on a megapixel image costs three floats of scale memory, not three million.

Mini-example — this operation in isolation

You can see the stride-0 view directly: `torch.tensor([1.,2.,3.,4.]).expand(3,4)` reports `stride()` of (0, 1) and shares storage with the original. Writing into such an expanded view is forbidden precisely because many logical cells alias the same physical byte — the 0 stride is the giveaway that no copy was made.

8 Step 6 — The silent trap: (3, 1) vs (1, 3) is an outer product

Broadcasting’s danger is that it often *succeeds* when you wanted it to fail. Suppose you have a length-3 vector $v = [1, 2, 3]$ and you want the elementwise sum $v + v = [2, 4, 6]$. If a stray `reshape`, `unsqueeze`, or a `keepdim=True` reduction leaves one copy as a column (3, 1) and the other as a row (1, 3), the rule does *not* error — it broadcasts both size-1 axes and hands you a 3×3 matrix:

$$\underbrace{\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}}_{(3,1)} + \underbrace{\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}}_{(1,3)} = \begin{bmatrix} 2 & 3 & 4 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix} \quad (3,3), \quad \text{not} \quad [2, 4, 6].$$

This is the *outer* sum: cell $(i, j) = v_i + v_j$. The code runs, no exception is raised, and a downstream `.mean()` or `.sum()` happily reduces the wrong 3×3 tensor to a plausible-looking scalar. The bug surfaces only as a wrong *number*, often much later.

Watch out

The defence is to *assert shapes*, not just dtypes. Whenever an elementwise op feeds a loss, print or assert the result shape: `assert out.shape == (3,)`. An accidental (n, n) where you expected $(n,)$ is the classic “my loss looks fine but the gradients are nonsense” bug. The same trap appears as $(n, 1)$ vs $(n,)$, which pads to $(1, n)$ and again explodes into (n, n) .

9 Broadcasting, at a glance

#	Pair of shapes	Result	What happened
1	$(3, 1) + (1, 4)$	$(3, 4)$	both size-1 axes stretch (outer sum)
2	$(2, 2, 3) \times (3,)$	$(2, 2, 3)$	$(3,)$ pads to $(1, 1, 3)$, two axes stretch
3	$(4, 3) + (3,)$	$(4, 3)$	$(3,)$ pads to $(1, 3)$, batch axis stretches
4	$(3,) + (4,)$	error	$3 \neq 4$ and neither is 1
5	$(3, 1) + (1, 3)$	$(3, 3)$	<i>silent</i> outer product — usually a bug

Rows 1–3 are the operations you *want*; row 4 is the error that protects you; row 5 is the success that doesn’t. Knowing which is which is most of what “getting good at shapes” means.

10 Equation cheat-sheet

Alignment	compare shapes right-to-left (last axis first)
Left-pad	shorter shape gets leading 1s: $(C,) \rightarrow (1, \dots, 1, C)$
Compatible axis	$a = b$ or $a = 1$ or $b = 1$
Output dim	$\text{out_dim} = \max(a, b)$
Fails when	$a \neq b$ and $a \neq 1$ and $b \neq 1$
Stretch in memory	set stretched axis stride = 0 (no copy)
Stretched view	many logical cells alias one physical element
Outer-product trap	$(n, 1)$ vs $(1, n) \Rightarrow (n, n)$, not elementwise

Appendix A Reference implementation

Every number above is reproducible from the following program (NumPy; PyTorch gives identical results and is used to show the stride-0 views). Change the shapes to explore.

```
import numpy as np, torch

# --- Step 1: (3,1)+(1,4) -> (3,4) outer sum ---
a = np.array([[10.],[20.],[30.]])      # (3,1)
b = np.array([[1.,2.,3.,4.]])          # (1,4)
print(a + b)                           # (3,4): rows 11..14, 21..24, 31..34

# --- Step 2: per-channel scale (2,2,3)*(3,) ---
img = np.arange(12, dtype=float).reshape(2,2,3)
scale = np.array([0.5, 1.0, 2.0])      # (3,) -> pads to (1,1,3)
print(img * scale)                     # out[1,1] = [4.5, 10, 22]

# --- Step 3: bias add (4,3)+(3,) ---
A = np.arange(12, dtype=float).reshape(4,3)
bias = np.array([100., 200., 300.])    # (3,) -> pads to (1,3)
print(A + bias)                        # R[3] = [109, 210, 311]

# --- Step 4: (3,)+(4,) FAILS ---
try:
    np.array([1.,2.,3.]) + np.array([1.,2.,3.,4.])
except ValueError as e:
    print("numpy:", e)                  # could not be broadcast (3,) (4,)
try:
    torch.tensor([1.,2.,3.]) + torch.tensor([1.,2.,3.,4.])
except RuntimeError as e:
    print("torch:", e)                  # ... non-singleton dimension 0

# --- Step 5: stretching == stride 0 (no copy) ---
bt = torch.tensor([1.,2.,3.,4.])        # (4,) stride (1,)
print(bt.expand(3,4).stride())          # (0, 1) <- new axis has stride 0
at = torch.tensor([[10.],[20.],[30.]]) # (3,1) stride (1,1)
print(at.expand(3,4).stride())          # (1, 0) <- column axis held still

# --- Step 6: the silent (3,1) vs (1,3) outer-product trap ---
v = np.array([1.,2.,3.])
print(v.reshape(3,1) + v.reshape(1,3)) # (3,3) outer sum, NOT [2,4,6]
print(v + v)                            # [2, 4, 6] <- what you meant

# output rule: out_dim = max(a, b)
for x, y in [(3,1),(1,4),(2,2),(4,4),(1,1),(5,1)]:
    print(x, y, "->", max(x, y))
```

— end of document —